

CA2

Object Oriented Programming

Banking Operations Application

Submitted to
Damien Kettle

Submitted by
Pratyush Kakkar
(20022933)

Date
14th April, 2025

Table of Contents

Banking Operations Application	0
Introduction.....	2
System Design.....	3
Tables	3
Stored Procedures:.....	4
Account Management.....	4
Transaction	5
Transaction History	5
Graphical User Interface.....	7
UI Styling & Consistency	7
Window Components:.....	8
Client-Side Validation.....	11
Business Layer.....	12
1. ExportXML Class.....	12
2. Transfer Class.....	13
3. UserAccount Class	14
4. UserTransactions Class.....	15
Data Access Layer.....	16
Key Components:.....	16
1. DAO (Data Access Object) Base Class	16
2. AccountData Class	16
3. TransactionData Class	17
Testing.....	18
1. Test Project Configuration	18
2. Test Coverage Highlights.....	19
1. Business Logic Verification	19
2. Data Layer Validation	19
3. Presentation Layer Checks	19
Conclusion	20
Bibliography	21

Banking Operations App

Introduction

This report describes the design and development of a desktop banking application prototype, created as part of the **Object Oriented Programming** (OOP) module assessment. The application is meant for use by bank staff to carry out basic banking tasks while following standard software development practices.

The system is built using **C#** and Windows Presentation Foundation (**WPF**) to provide a responsive and clear user interface. **Microsoft SQL Server** is used as the database to store and manage data reliably. The application follows a three-layer structure: Presentation, Business, and Data. This setup keeps each part of the system separate, making it easier to maintain and update in the future. Each part works on its own but also connects well with the rest of the system.

The application allows staff to create, edit, and manage customer accounts. It also supports key financial tasks like deposits, withdrawals, and transfers between accounts. Validation checks are included to help prevent mistakes and stop unauthorized actions. The system also keeps a full record of all transactions to support transparency and follow banking rules. Data is saved in a structured database, and XML is used to export account details in a standard format.

To make sure everything works well, the system has been tested using both unit and integration tests. The development followed object oriented principles such as encapsulation, inheritance, and polymorphism, which help make the code more organized and reusable.

System Design

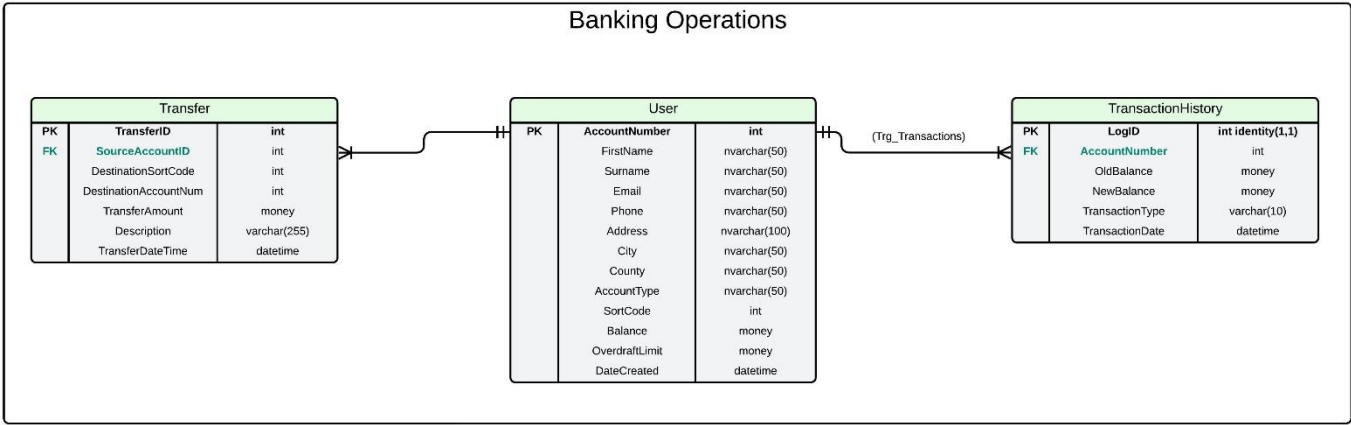


Figure 1 Entity Relationship Diagram

The Service-Based database (DB.mdf) consists of three tables: **User**, **Transfers**, and **TransactionHistory**, User Stored Procedures and a Trigger.

Tables

1. User Table

- **Primary Key:** AccountNumber (unique identifier for each user).
- **Unique Constraints:**
 - Email and Phone are enforced as unique to prevent duplicate entries.
- **Default Values:**
 - SortCode defaults to **101010** (as per business requirements).
 - OverdraftLimit defaults to **0.0**, ensuring new accounts start with no overdraft unless modified.
 - DateCreated automatically records the creation timestamp using GETDATE().

2. Transfers Table

- **Primary Key:** TransferID (auto-incremented in steps of 5, starting from 100).
- **Foreign Key:** SourceAccountID references User.AccountNumber.
- **Check Constraint:** Ensures TransferAmount must be greater than **0**, preventing invalid transactions.
- **Default Values:**
 - TransferDateTime defaults to the current timestamp (GETDATE()).
 - Description has a default empty string (' ').

3. TransactionHistory Table

- **Primary Key:** LogID (auto-incremented for each transaction log).
- **Automatic Logging via Trigger:**
 - The trg_Transactions trigger on the User table logs every balance change.
 - It captures:
 - OldBalance and NewBalance
 - Transaction type (**CREDIT** if balance increases, **DEBIT** if it decreases)
 - Timestamp of the transaction (GETDATE())

Stored Procedures:

Account Management

1. uspAddAccount

- Creates new bank accounts
- Takes parameters for all account details including:
 - Account number, name, contact info
 - Address details
 - Account type and initial balance
- Inserts complete record into User table
- Returns 0 on success

2. uspUpdateAccDetails

- Modifies existing account information
- Updates editable fields:
 - Email, phone number
 - Full address (street, city, county)
- Uses AccountNumber to identify record
- Returns 0 on completion.

3. uspGetAccDetails

- Retrieves full account information
- Accepts AccountNumber as input parameter
- Returns all fields for single account

4. UspGetAccNums

- Retrieves basic account information
- Returns all accounts with all fields
- Used for account listings

Transaction

1. uspAddTransferRecord

- Records money transfers between accounts
- Stores:
 - Source and destination account details
 - Transfer amount and timestamp
 - Optional description
- Defaults description to empty string if not provided

2. uspUpdateBalance

- Modifies account balances
- Takes AccountNumber and new Balance
- Directly updates User table
- Triggers transaction logging

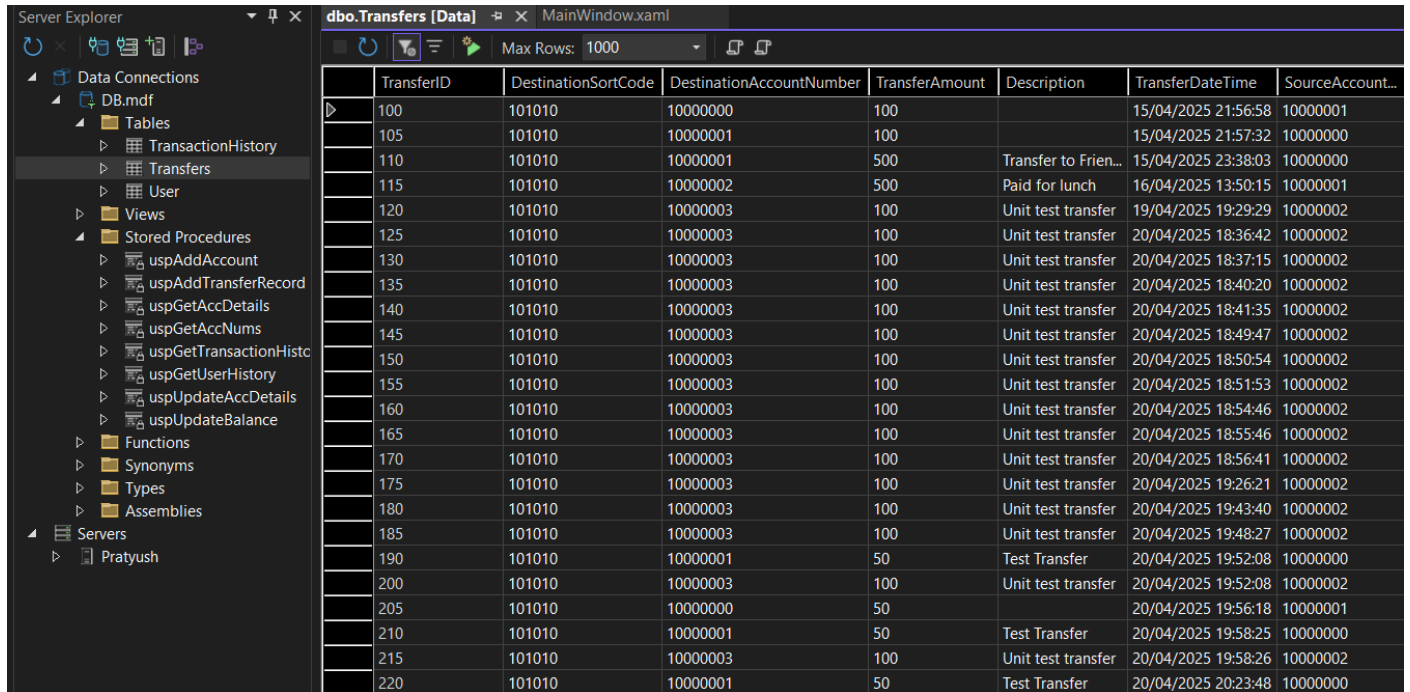
Transaction History

1. uspGetTransactionHistory

- Retrieves complete transaction log
- Returns all records from TransactionHistory
- No filtering parameters

2. uspGetUserHistory

- Gets transaction history for specific account
- Filters by AccountNumber
- Returns all related transactions



TransferID	DestinationSortCode	DestinationAccountNumber	TransferAmount	Description	TransferDateTime	SourceAccount...
100	101010	10000000	100		15/04/2025 21:56:58	10000001
105	101010	10000001	100		15/04/2025 21:57:32	10000000
110	101010	10000001	500	Transfer to Frien...	15/04/2025 23:38:03	10000000
115	101010	10000002	500	Paid for lunch	16/04/2025 13:50:15	10000001
120	101010	10000003	100	Unit test transfer	19/04/2025 19:29:29	10000002
125	101010	10000003	100	Unit test transfer	20/04/2025 18:36:42	10000002
130	101010	10000003	100	Unit test transfer	20/04/2025 18:37:15	10000002
135	101010	10000003	100	Unit test transfer	20/04/2025 18:40:20	10000002
140	101010	10000003	100	Unit test transfer	20/04/2025 18:41:35	10000002
145	101010	10000003	100	Unit test transfer	20/04/2025 18:49:47	10000002
150	101010	10000003	100	Unit test transfer	20/04/2025 18:50:54	10000002
155	101010	10000003	100	Unit test transfer	20/04/2025 18:51:53	10000002
160	101010	10000003	100	Unit test transfer	20/04/2025 18:54:46	10000002
165	101010	10000003	100	Unit test transfer	20/04/2025 18:55:46	10000002
170	101010	10000003	100	Unit test transfer	20/04/2025 18:56:41	10000002
175	101010	10000003	100	Unit test transfer	20/04/2025 19:26:21	10000002
180	101010	10000003	100	Unit test transfer	20/04/2025 19:43:40	10000002
185	101010	10000003	100	Unit test transfer	20/04/2025 19:48:27	10000002
190	101010	10000001	50	Test Transfer	20/04/2025 19:52:08	10000000
200	101010	10000003	100	Unit test transfer	20/04/2025 19:52:08	10000002
205	101010	10000000	50		20/04/2025 19:56:18	10000001
210	101010	10000001	50	Test Transfer	20/04/2025 19:58:25	10000000
215	101010	10000003	100	Unit test transfer	20/04/2025 19:58:26	10000002
220	101010	10000001	50	Test Transfer	20/04/2025 20:23:48	10000000

Figure 2 Data of Table Transfers

Graphical User Interface

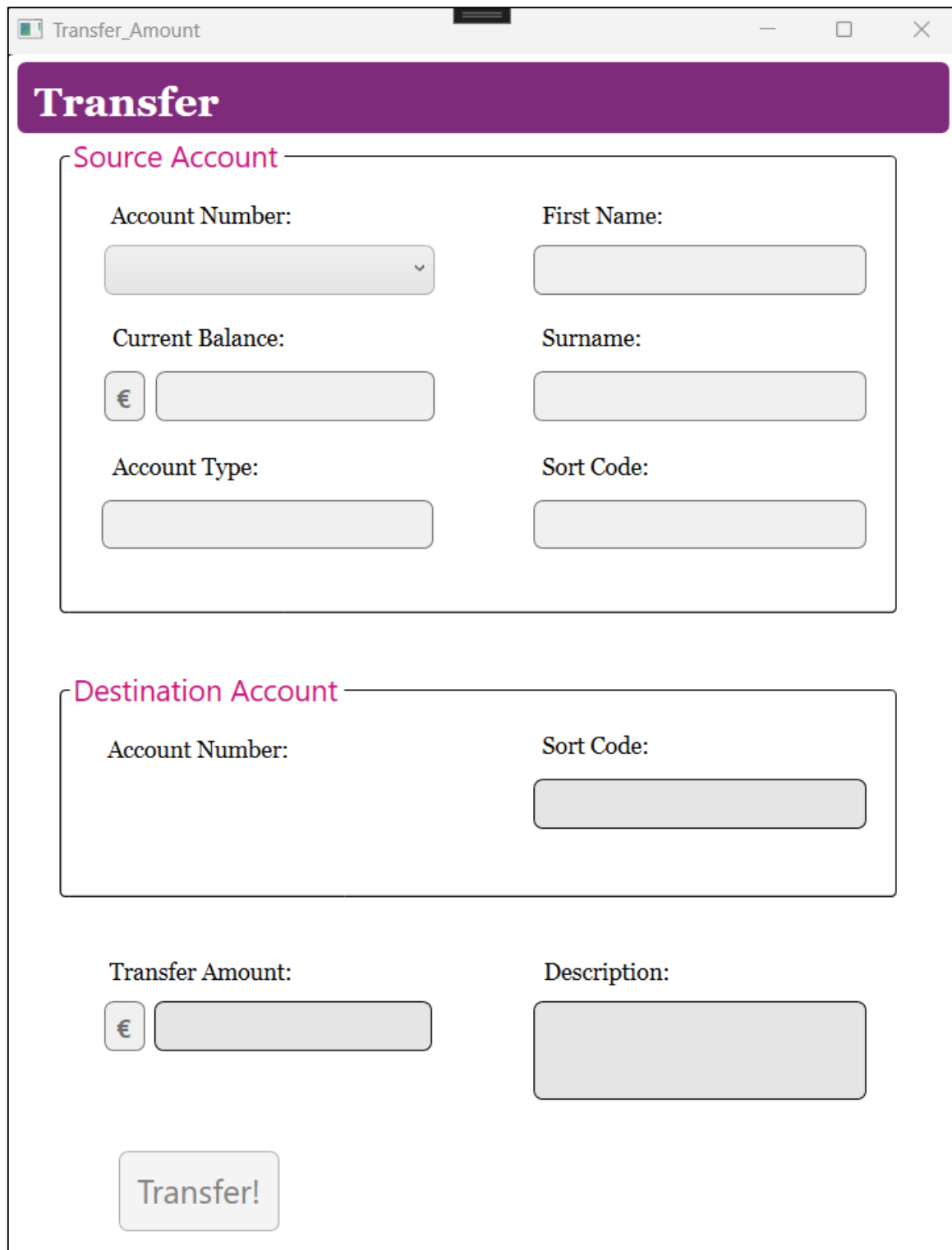
The banking application's user interface is built using WPF in Visual Studio 2022, featuring several windows that handle different banking operations. The presentation layer includes client-side validation to ensure data integrity before processing.

UI Styling & Consistency

To maintain a uniform and professional appearance across the application:

- **Styles were defined in App.xaml under** Application.Resources, allowing centralized control over UI elements such as:
 - **Labels, TextBoxes, Buttons, and Menus** - ensuring consistent fonts, colours, and spacing.
 - **Button Formatting:**
 - Underlined text in buttons was achieved using `<TextBlock>` with `<Underline>`, improving readability while maintaining a clean design.
- **Colour Palette:**
 - Primary: **Deep teal** (#00856a)
 - Secondary: **Regal purple** (#7f2b7b)
 - Background: Warm off-white (#fffcfafa)

Window Components:



The image shows a window titled "Transfer_Amount" with a purple header bar containing the word "Transfer" in white. Below the header, there are two main sections: "Source Account" and "Destination Account".

Source Account

Account Number:	First Name:
Current Balance:	Surname:
€	
Account Type:	Sort Code:

Destination Account

Account Number:	Sort Code:

Transfer Amount: €

Description:

Figure 3 Transfer Account Window

1. MainWindow.xaml (Login Window)

- Entry point of the application
- Contains username and password fields
- Implements secure password masking
- Features login button with validation

2. New Account Window

- Form for creating new bank accounts
- Collects all required account information:
 - Personal details (name, contact info)
 - Address information
 - Account type selection
 - Initial deposit amount

3. Edit Account Window

- Allows modification of existing accounts
- Displays current account information
- Restricts editing of immutable fields
- Validates all input changes

4. Transfer Amount Window

- Handles funds transfer between accounts

- Captures:
 - Source and destination account details
 - Transfer amount
 - Optional reference note
- Includes balance verification

5. User Transaction Window

- Processes deposits and withdrawals
- Simple interface with amount input
- Transaction type selector
- Balance update confirmation

6. View Transactions Window

- Displays transaction history
- Filterable by account and date range
- Export functionality to XML
- Clear data presentation

7. Under Construction Window

- Placeholder for future features
- Appears when accessing unimplemented functionality
- Maintains application flow

All changes made to the database by any window reflect in the Data Grid on the Main Window:

```
var mainWindow = Application.Current.Windows.OfType<MainWindow>().FirstOrDefault();
mainWindow?.RefreshDataGrid();
```

Client-Side Validation

The InputValidation class provides comprehensive validation for all user inputs:

1. Login Validation

- **ValidateUsername():** Ensures alphabetic characters only
- **ValidatePassword():** Requires minimum 8 characters
- **CheckCredentials():** Verifies against hardcoded admin credentials

2. Account Information Validation

- **IsValidName():** Validates personal names with allowed special characters
- **IsValidEmail():** Basic email format verification
- **IsValidPhone():** Strict 9-digit number requirement
- **IsValidAddress():** Address format with permitted special characters

3. Financial Validation

- **IsValidAccountNumber():** 8-digit numeric validation
- **IsNumericOrDot():** Numeric input for amounts
- **IsValidSortCode():** 6-digit numeric validation

4. Form Control

- **EnableButton():** Controls button states based on validation
- Consistent validation feedback across all forms

Business Layer

The business layer (BIZ) serves as the core processing engine of the banking application, implementing all financial logic and data transformation between the presentation layer and data access layer. This layer consists of four main components that handle specific banking operations.

1. ExportXML Class

Responsibility: Handles XML serialization of account data for reporting and data exchange

Key Features:

- Combines account details with transaction history into structured XML
- Creates two distinct XML sections:
 - **UserDetails:** Contains all account information from the User table
 - **TransactionHistory:** Lists all transactions with complete details
- Uses LINQ-to-XML for clean document generation
- Testable design with injectable database connections

Processing Flow:

1. Retrieves account details via AccountData DAL class
2. Fetches transaction history via TransactionData DAL class
3. Constructs hierarchical XML document using XDocument
4. Returns complete XML structure to caller

2. Transfer Class

Responsibility: Manages all funds transfer operations between accounts

Core Logic:

- Validates transfer amount against available balance + overdraft
- Processes both internal (same bank) and external transfers
- Maintains atomic transaction integrity
- Automatically records transfer details

Transfer Process:

1. Verifies sufficient funds in source account
2. Deducts amount from source account
3. For internal transfers (sort code 101010):
 - Credits destination account
 - Updates both account balances
4. Creates permanent transfer record
5. Returns success/failure status

Business Rules:

- Transfer amount must be positive
- Source account must have sufficient available balance
- External transfers don't modify destination balance
- All transfers are timestamped

3. UserAccount Class

Responsibility: Manages bank account lifecycle operations

Account Operations:

- **Creation:** Handles new account setup with all required fields
- **Modification:** Updates editable account information
- **Data Integrity:** Maintains relationship between properties and database

Key Characteristics:

- Separate constructors for create vs update operations
- Immutable properties for critical fields (AccountNum)
- Clear separation between:
 - Core account information (name, contact details)
 - Financial properties (balance, overdraft)
 - Address information
- Test support through connection string injection

4. UserTransactions Class

Responsibility: Processes individual account transactions

Transaction Types:

- **Deposits:**
 - The amount must be positive
 - Immediately increases account balance
- **Withdrawals:**
 - Validates against available balance + overdraft
 - Processes only if sufficient funds available

Business Rules Enforcement:

- Negative amounts rejected
- Withdrawals respect overdraft limits
- Balance updates are atomic operations
- All changes trigger transaction logging

Data Access Layer

The **Data Access Layer (DAL)** is a **class library** that handles all database interactions for the banking application. It follows a **structured and modular** approach, ensuring secure and efficient communication with the **MS SQL Server** database. The DAL abstracts SQL operations, allowing the business layer to work with simple method calls rather than direct database queries.

Key Components:

1. DAO (Data Access Object) Base Class

- **Responsibility:** Manages database connections.
- **Features:**
 - Establishes and closes SQL connections.
 - Retrieves the connection string from `ConfigurationManager` (App.config).
 - Ensures connections are properly opened and closed to prevent leaks.

2. AccountData Class

- **Responsibility:** Handles **account-related** database operations.
- **Key Methods:**
 - **addAccount()** → Inserts a new bank account using the stored procedure `uspAddAccount`.
 - **updateAccount()** → Modifies existing account details via `uspUpdateAccDetails`.
 - **PopulateAccNums()** → Retrieves all account numbers (`uspGetAccNums`).
 - **PopulateAccountDetails()** → Fetches full account details for a given account number (`uspGetAccDetails`).

3. TransactionData Class

- **Responsibility:** Manages **transaction-related** database operations.
- **Key Methods:**
 - **updateBalance()** → Updates an account's balance (uspUpdateBalance).
 - **AddTransferRecord()** → Logs a new funds transfer (uspAddTransferRecord).
 - **GetTransactionHistory()** → Retrieves all transactions (uspGetTransactionHistory).
 - **GetUserHistory()** → Gets transaction history for a specific account (uspGetUserHistory).

Testing

The banking application employs a **structured testing approach** using **MSTest** to validate functionality across all layers. The test project is configured for **.NET 8.0** and integrates with the main application through project references to **Banking Operations App**, **BIZ (Business Layer)**, and **DAL (Data Access Layer)**.

1. Test Project Configuration

The .csproj file defines the testing environment with:

- **Testing Framework:** **MSTest.Sdk v3.6.4**

Dependencies:

- **System.Data.SqlClient** (v4.9.0) for database connectivity
- **System.Configuration** (v9.0.4) for connection string management
- **Target Platform:** **AnyCPU** for cross-platform compatibility

Language Features:

- Enabled implicit usings for cleaner test code
- Nullable reference types for better type safety

The test suite connects to the **local SQL Server database** (MSSQLLocalDB) using an attached MDF file.

2. Test Coverage Highlights

1. Business Logic Verification

- Fund transfer validation (sufficient/insufficient balance cases)
- Account operations (creation, updates)
- Transaction processing (deposits/withdrawals)
- XML export functionality

2. Data Layer Validation

- Database connection management
- CRUD operation testing
- Transaction logging

3. Presentation Layer Checks

- Input validation rules
- Data structure integrity
- UI enumeration consistency

The test methods follow **Arrange-Act-Assert** pattern and include both **positive and negative test cases**.

Database tests use a **real SQL Server instance** for integration testing.

Conclusion

The development of this banking application provided valuable insights into **object-oriented programming** and **3 layered software architecture**. Key learnings from the project included:

1. **Layer Separation** – The project reinforced the importance of proper component organization, with validation logic placed in the **presentation layer** while business rules remained isolated in their dedicated layer.
2. **GUI Design** – The use of **App.xaml** for centralized styling demonstrated an effective approach to maintaining visual consistency. The interface design took inspiration from Irish banking applications to ensure a professional and user-friendly experience.
3. **Technical Implementation** – Several new skills were developed, including:
 - Generating **XML reports** programmatically in C#
 - Implementing **database triggers** for automated transaction auditing
 - Conducting **comprehensive testing** across presentation, business, and data layers
4. **Database Management** – Working with **stored procedures, transactions, and constraints** provided practical experience in secure data handling for financial systems.

The **three-tier architecture** (Presentation, Business, and Data Access Layers) proved effective in creating the solution.

Bibliography

1. **Kettle, D.** (2025) Object-Oriented Programming (B7IS111_2425_TMD1) [Lecture recording]. Dublin Business School. Available at: <https://elearning.dbs.ie/mod/book/view.php?id=1610365> .
2. **Gaddis, T.** (2016) Starting out with Java: from control structures through objects. 6th edn. Pearson Education. Available at: <https://ebookcentral.proquest.com/lib/dbsie/detail.action?docID=5185678>.
3. **Deitel, P.** and **Deitel, H.** (2020) Java: how to program - late objects version. 11th edn. Pearson Education. Available at: <https://ebookcentral.proquest.com/lib/dbsie/detail.action?docID=5843853>.
4. **Savitch, W.** and **Mock, K.** (2016) Absolute Java. 6th edn. Pearson Education. Available at: <https://ebookcentral.proquest.com/lib/dbsie/detail.action?docID=5186215>.